

Summary

The semiconductor industry responded to customer demands in the 1980s for a configurable logic circuit that could be used to rapidly implement high-speed logic circuitry yet be reconfigurable as the design matures or system specifications change. These **complex programmable logic devices** (CPLDs) and **field-programmable gate arrays** (FPGAs) use tens of thousands of configurable logic blocks, memories, and I/O pins to provide a rich fabric for engineers to design complex systems. Each configurable logic block usually includes a RAM-based truth table (a look-up table) with configurable inputs, a D-type flip-flop, and support logic to enable interconnection to other logic blocks. Modern design automation tools allow engineers to use schematics and hardware description languages to describe designs. Synthesis tools evaluate designs for errors, create logic equations, generate the look-up-table data for storage in the LUT RAMs, and compile wiring configuration files used to store configuration binary patterns in the CPLD or FPGA configuration memory. The Altera MAX10 FPGA **logic element (LE)** is shown in Figure 1.

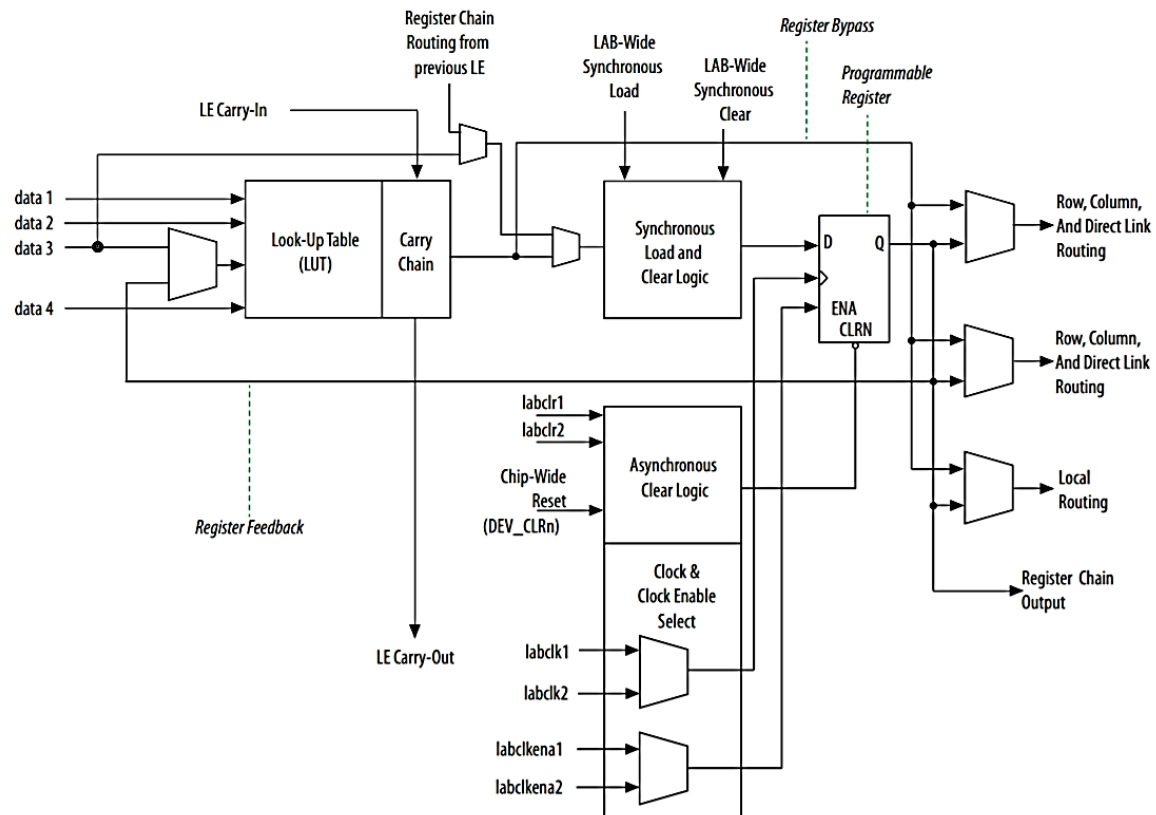


Figure 1: The MAX10 Configurable Logic Element

Altera markets the MAX10 as a set of chips with differing amounts of configurable logic elements. Each chip begins with 10MXX, where XX represents the number of logic elements in thousands. The current MSOE laboratory board uses a 10M50 device with 50,000 logic elements. **In this laboratory**, students implement a system level computer architecture with an ARMv4 single-cycle processor, memory-mapped I/O devices, and a reset synchronizer. This computer is configured in the Altera 10M50DAF484C7G FPGA on the DE10-Lite laboratory board.

Project Management

This project should be completed with the CPE1510 basic single-cycle processor. **Ensure** you have a backup archive of your working single-cycle processor project **before beginning** the modifications required for this laboratory.

- Use Project → Archive to make a backup archive.

Read this entire document before beginning your work to help you understand the changes you will make.

Read carefully and when you re-read and begin your work, work slowly.

System Architecture

All computers contain the five components listed by John Von Neumann in the paper he wrote in 1945 entitled *First Draft of a Report on the EDVAC*. These five components are **input**, **output**, **arithmetic circuits**, **memory**, and **control circuits**. They are generally organized into the system architecture shown in Figure 2.

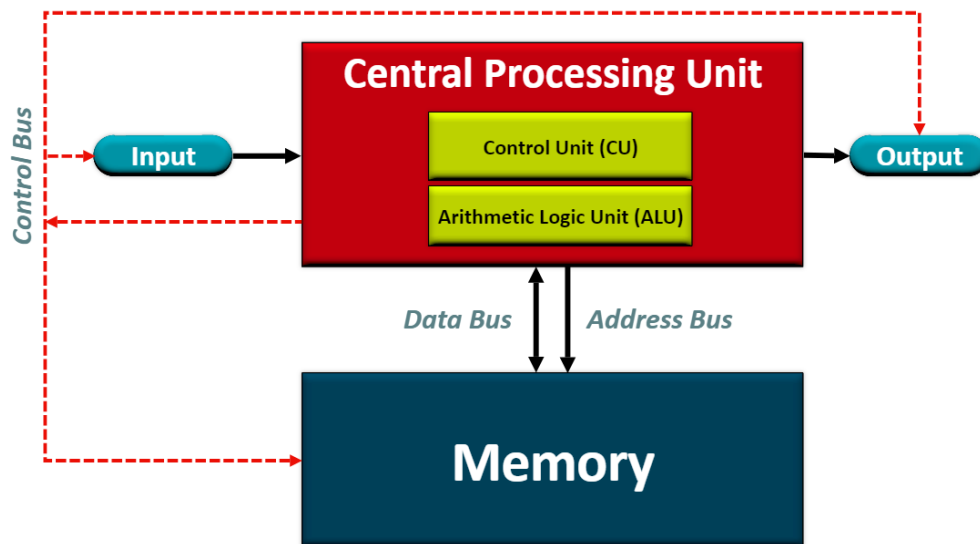


Figure 2: The Basic Organization of a Stored-Program Computer

In this model, computer instructions are stored as binary numbers in memory. The processor fetches the next instruction from memory, retrieves stored numbers as data, completes the calculation, and stores the calculated result back to memory.

- The **Princeton organization** shares one memory for instructions and data through a single connection to the processor. This single connection prevents simultaneous instruction and data movement. This bottleneck limits performance. It is an optimization of circuit size over speed.
- The **Harvard organization** uses two separate memories to enable simultaneous instruction and data movement and thus improved performance. This is an optimization of speed over size.

Inputs and outputs are peripheral devices that provide ways to interact with users. Input devices and output devices are accessed using one of two standard methods.

- A **memory-mapped I/O device** appears as one or more memory locations in the computer memory. These memory locations correspond to control, data, and status registers in the I/O device. Access to these registers is made through the standard load and store instructions. The system architecture contains an address decoder module that monitors the memory address and asserts the appropriate control signals on the registers of the I/O device whenever the device is accessed using a load or store instruction. The ARM instruction set specifies that I/O devices must be memory-mapped. Accessing I/O devices exactly like memory keeps to the RISC philosophy of **regularity implies a simple and smaller design**.
- A **port-mapped I/O device** is accessed using special I/O instructions defined in the instruction set. The device is accessed using these special input and output instructions. The Intel x86 instruction set includes the **in** and **out** instructions. Added instructions increase instruction set complexity.

In this laboratory, three **memory-mapped** output devices and one two **memory-mapped** input devices are added to the ARM single-cycle processor. This system architecture is then configured in the MAX10 FPGA on the DE10-Lite laboratory board. The system architecture is shown in Figure 3, and the memory and I/O devices are documented in the memory-map of Table 1.

Table 1: The CPE1510 ARM Computer Devices

DEVICE	TYPE	MEMORY ADDRESS	BEHAVIOR
LED	Output	0x000000E0	Displays the lower ten bits of its 32-bit data register.
MOTORS	Output	0x000000E4	Lower five bits control BEEP LEN LDIR REN RDIR
SEG7	output	0x000000E8	Displays the lower five nibbles of its 32-bit wide data register.
SENSORS	Input	0x000000EC	
SLIDER	input	0x000000F0	Provides the lower ten bits of its 32-bit wide data register.
MAIN MEMORY	mem	0x00000000 – 0x0000001F	A 32-location main memory used for general-purpose data storage. The size is kept small to enable faster simulation.

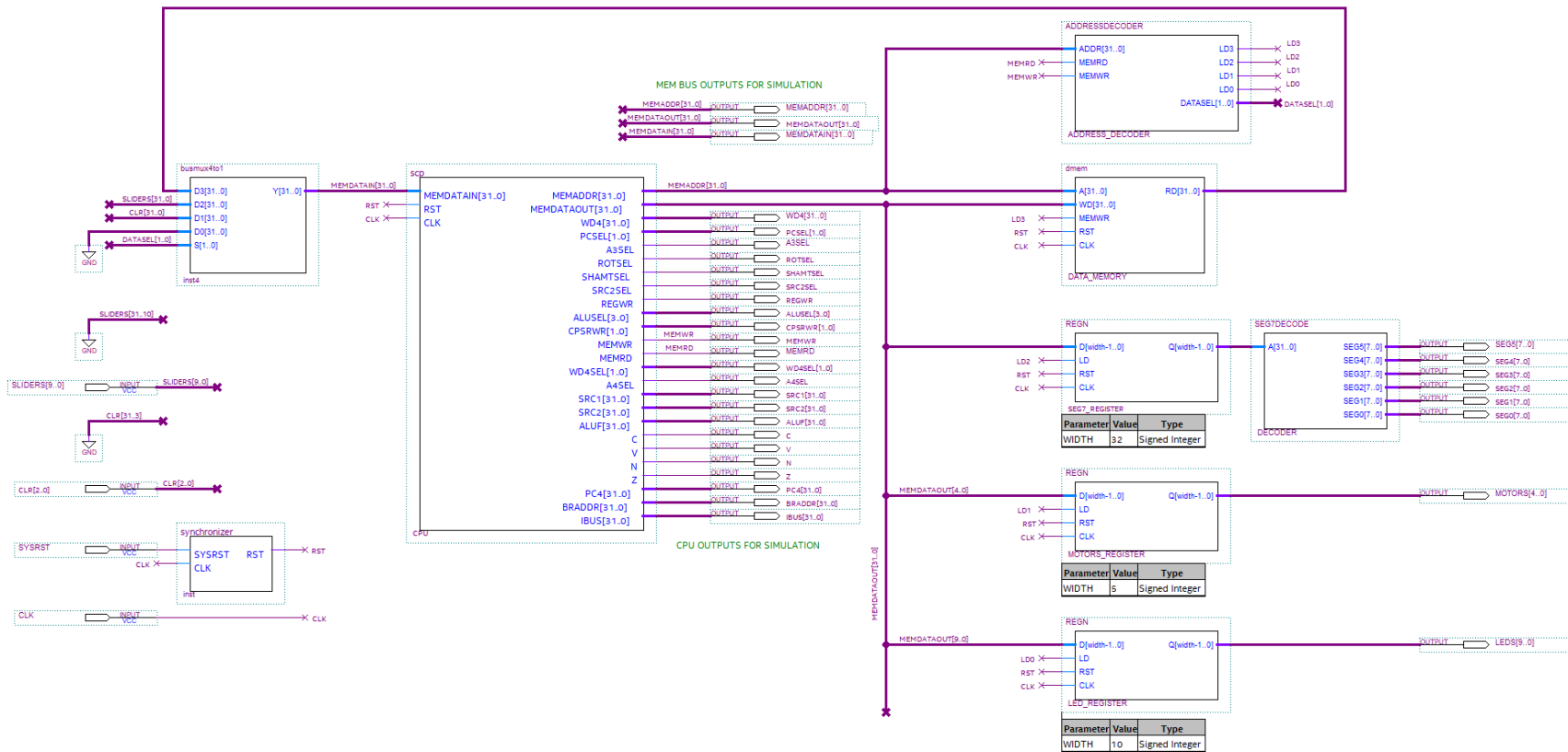


Figure 3: The CPE1510 Robot Computer

Modifying the Processor

The basic CPE1510 single-cycle processor completed in the previous laboratory included a data memory circuit stage. Adding memory-mapped I/O devices requires generalizing the circuit to allow additional memory-mapped components to provide data to the WD4 multiplexer.

All laboratory exercises are modifications and additions within your single-cycle processor project.

1. **Replace** the data memory component in your single-cycle processor with a memory address output bus, a memory data output bus, and a memory data input bus as shown in Figure 4. **Note** all you are doing is removing DMEM and adding these busses as shown between the execute and WB4 mux. Green text can be added using the schematic editor text toolbar icon.

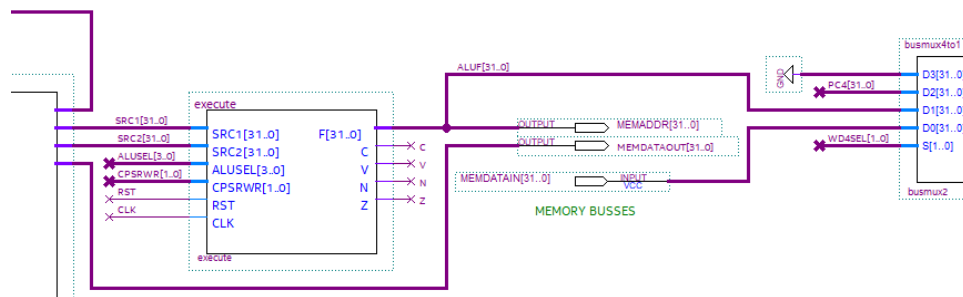


Figure 4: Memory busses replace the memory circuit in the single cycle processor

2. **Replace** the program counter in the fetch stage with a VHDL generic width register as shown in Figure 5. **Set** the width parameter to 32 bits as shown in Figure 5. **Use** the provided REGN skeleton file.

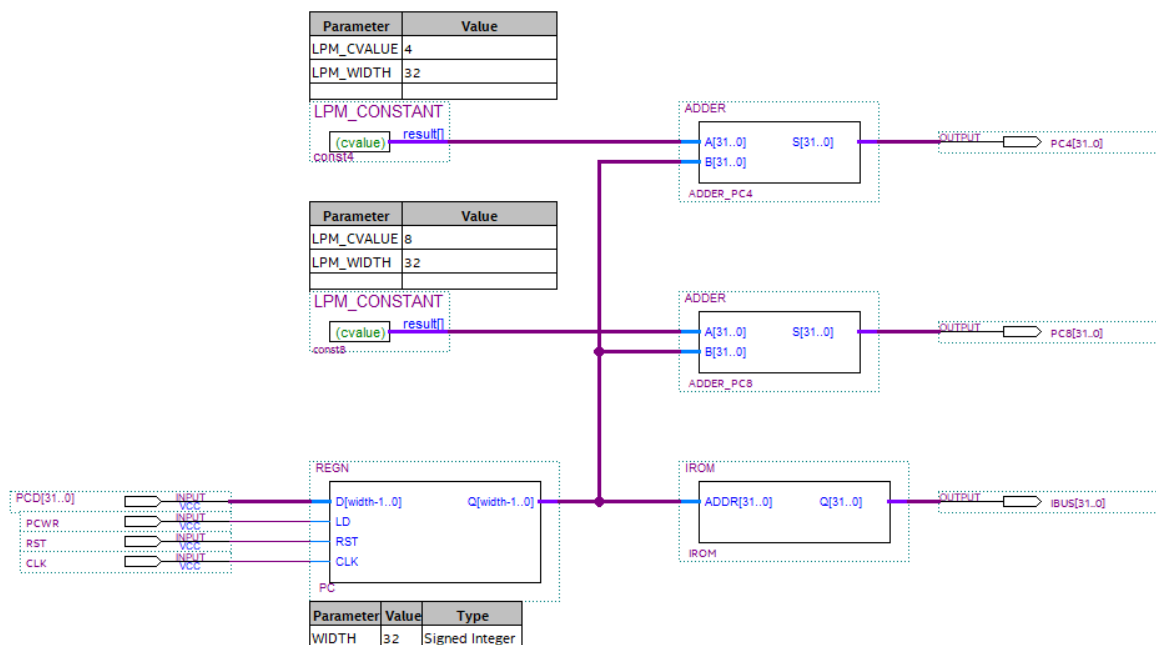


Figure 5: The program counter in fetch implemented using a generic width VHDL description

3. **Create** a new block diagram symbol for the single-cycle processor using File → Create/Update → Create Symbol File. This component will be integrated into a system level diagram. When you later place the component in the system level diagram it will look something like Figure 6, but the signals may be in a different order based on how you placed inputs and outputs in your schematic diagram.

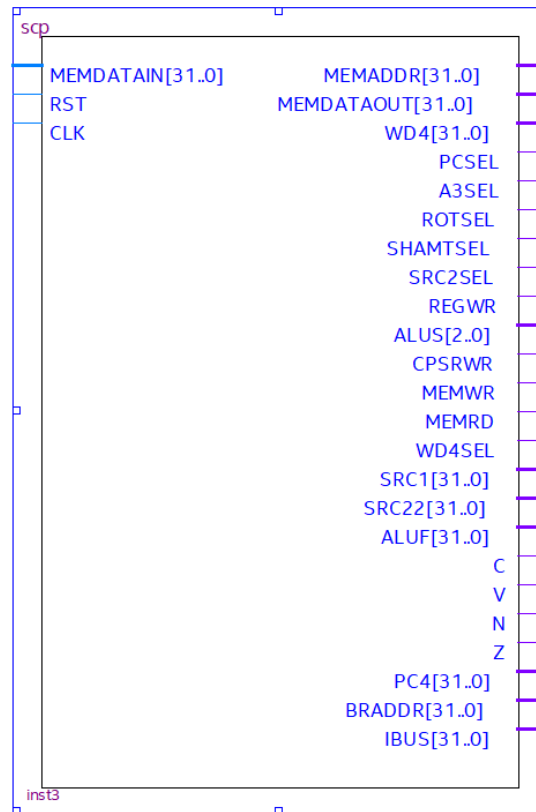


Figure 6: The Single-Cycle Processor Symbol

Seven-Segment Decoder (SEG7DECODE.VHD)

Complete the provided skeleton VHDL code for a seven-segment display decoder that generates six output busses. Each output bus carries the voltages to turn on the display LEDs of one seven-segment display. The decoder accepts a 32-bit number and displays the lower six **hex nibbles** of the number. The DE10-Lite has seven-segment LEDs that are active-low. In each output byte, bits of the byte correspond to the similarly numbered LED in Figure 7. The decimal point (DP) LED is bit 7 of the bus. The decimal point should never be on. **Create** the component symbol using **File → Create/Update → Create Symbol File**.



Figure 7: The DE10-Lite 7-segment LED arrangement

Address Decoder (ADDRESSDECODER.VHD)

Complete the skeleton VHDL code for the system architecture address decoder. This component requires the processor address bus, the processor MEMRD control signal, and the processor MEMWR control signal as inputs. It outputs individual load signals for memory and the I/O device registers. It also controls the data input multiplexer. The behavior is summarized in Table 2. **Use** File → Create/Update → Create Symbol Files to make its symbol.

- **Examine ADDR row one.** If the memory address is in the range of the data memory and the MEMRD and MEMWR control signals show the processor is executing STR then the address decoder commands the data memory to store data by asserting LD3.
- **Examine ADDR row four.** If the memory address is the address of the LED register and the processor is executing STR then the address decoder commands the LED register to store data by asserting LD0.
- **Examine ADDR row five.** If the memory address is in the range of the data memory and the processor is executing LDR then the address decoder commands no changes on the outputs and routes the memory output through the data input bus multiplexer.
- **Examine ADDR row seven.** If the memory address is the address of the SLIDER SWITCHES and the processor is executing LDR then the address decoder commands no changes on the outputs and routes the slider switch values through the data input multiplexer.

Table 2: The Address Decoder Truth Table

INPUTS				OUTPUTS				
ADDR	MEMRD	MEMWR	ACTION	MEM LD3	SEG7 LD2	MOTOR LD1	LEDS LD0	MUX DATASEL
0x00000000 – 0x0000001F	1	0	STR TO MEM	0	1	1	1	0
0x000000E8	1	0	STR TO SEG7	1	0	1	1	0
0x000000E4	1	0	STR TO MOTORS	1	1	0	1	0
0x000000E0	1	0	STR TO LEDS	1	1	1	0	0
0x00000000 – 0x0000001F	0	1	LDR FROM MEM	1	1	1	1	3
0x000000EC	0	1	LDR FROM SENSORS	1	1	1	1	1
0x000000F0	0	1	LDR FROM SLIDERS	1	1	1	1	2

Reset Metastability Synchronizer (SYNCHRONIZER.BDF)

System resets occur during power-on stabilization, testing, and user operation. All register components use asynchronous reset earlier in the laboratory so that the computer can be reset even if the clock has stopped due to a failure. During normal operation, asynchronous signals, such as that from a user reset pushbutton, can violate the setup and hold times of registers because they occur too close to the edge of the register sample clock. This leads to a condition within the D flip-flop circuit known as **metastability**. During metastability, the flip-flop output did not have enough time to sample the input voltage before being commanded to stop by the clock edge. As a result, the output may be the incorrect value.

In this laboratory, a pushbutton from the DE10-Lite board provides the system reset signal. The DE10-Lite pushbuttons provide logic-0 when pushed. The computer will reset when the user pushes this button because the registers use asynchronous active-low reset. But, when the user releases the button, the logic-1 may occur too close to the clock edge and cause metastable behavior. A synchronizer can help realign the pushbutton release so that it does not violate the setup and hold times. The circuit is shown in Figure 8.

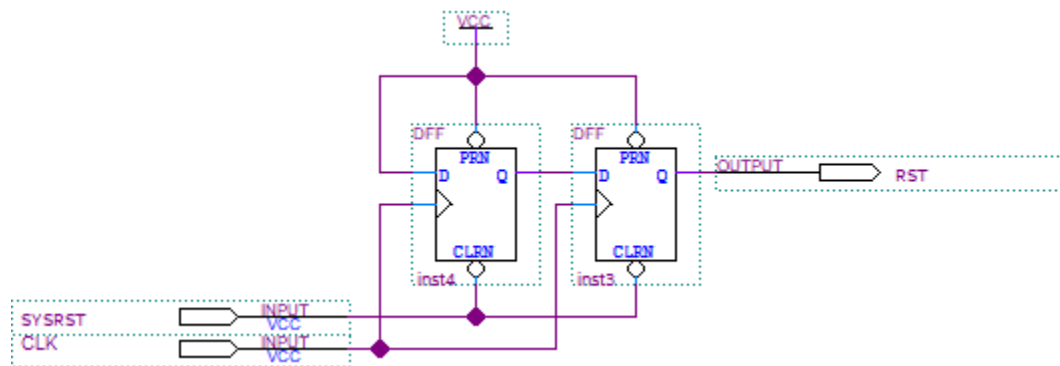


Figure 8: The Reset Pushbutton Synchronizer

The DFF components in Figure 8 have asynchronous reset behavior. Thus, a SYSRST pushbutton press will result in a near instantaneous logic-0 hitting the RST signal of the processor and I/O device registers. When the button is released, the first DFF may be left in a metastable state, but the second should correctly sample the logic-0 cleared in the first flip-flop. The hope is that on the next clock edge, the first DFF will correctly sample the logic-1 provided by VCC. The second DFF will sample whatever metastable value existed in the first DFF. This could have been a logic-1 or logic-0. In the best case, it was already a logic-1 and the reset ends throughout the system. In the worst case, the metastable state was logic-0 and a third clock cycle is required to propagate the VCC logic-1 out into the system and bring the registers out of reset. **Build** this synchronizer circuit as a new schematic block diagram within your project and then **use** File → Create/Update → Create Symbol Files to make its schematic block diagram symbol.

DE10-Lite Computer System (SYSTEM.BDF)

1. **Implement** the CPE1510 robot computer in Figure 3 as a schematic block diagram file called **system.bdf**.
2. **Use** Assignments → Device to ensure that you have the 10M50DAF484C7G selected as the device.
3. **Set** the **system.bdf** file as the top-level entity by right clicking **system.bdf** in project navigator.
4. **Synthesize** the design and correct any errors. Use the third triangle icon in the toolbar – the one with a small AND gate below it.
5. **Simulate** the computer using the original IROM. It should still function correctly. **Verify** correct operation. **Move** to step six **only after you correct all errors**.
6. **Assign** DE10-Lite chip pins to the system signals shown in Table 3. All other signals can be unassigned.

Table 3: DE10-Lite and Robot Pin Assignments

SIGNAL NAME	DE10-LITE ASSIGNMENT	USER MANUAL PAGE
CLK	MAX10 CLK1 50	24
SYSRST	KEY0	25
SLIDERS[9..0]	SLIDERS[9..0]	26
LEDS[9..0]	LEDR[9..0]	27
SEG0[7..0]	HEX0[7..0]	28
SEG1[7..0]	HEX1[7..0]	28
SEG2[7..0]	HEX2[7..0]	29
SEG3[7..0]	HEX3[7..0]	29
SEG4[7..0]	HEX4[7..0]	29
SEG5[7..0]	HEX5[7..0]	29
SIGNAL NAME	ROBOT ASSIGNMENT	ROBOT SILKSCREEN
MOTORS[4]	BEEP	AA11
MOTORS[3]	LEN	AB8
MOTORS[2]	LDIR	AB7
MOTORS[1]	REN	Y10
MOTORS[0]	RDIR	AB9
CLR[2]	COLLISION	AB17
CLR[1]	LEFT	AA12
CLR[0]	RIGHT	AA17

7. **Create** a new Synopsys Design Constraints file. This file contains specifications that we want the design automation tool to attempt to achieve. In the laboratory, the only constraint is a design that can clock using the 50MHz DE10-Lite oscillator. This means that the final circuit must propagate voltages across the logic cells and meet the setup and hold times of registers within a 20ns period. **Use** File → New → Synopsis Design Constraints File. **Add** this statement (use copy-paste) as the only line in the file and accept the default file name on save.

```
create_clock -period 20 [get_ports {clk}]
```

8. **Replace** the IROM in your design with the IROM provided as IROM2.vhd.
9. **Build** the design using Processing → Start Compilation. This may take up to **six or seven minutes** to complete. The FPGA compiler is attempting to place and route hundreds of system signals. If you receive an error message about too many signals and not enough pins, remove the MEMADDR, MEMDATAOUT, and MEMDATAIN simulation outputs from the schematic and rebuild.

10. **Program** the DE10-Lite board and test your design. **Try** setting the sliders to 10. Then press the reset button (KEY0) The result should be hexadecimal 0x37 on the seven-segment displays and a one on the LEDs. **Try** setting the sliders to fifteen. The result should be hexadecimal 0x78 on the seven-segment displays and a one on the LED. **Try** setting the sliders to 3. The result should be 0x06 on the seven-segment displays and a zero on the LEDs.

Demonstrations

1. A correct demonstration of IROM2 on the robot body earns a C-level grade.
2. A **B-level grade** is earned by writing your own main program that follows the black line and displays a count of the number of times it has crossed a speedbump on the seven-segment display.
3. An **A-level grade** is earned by writing your own main program with leaf subroutines.

- **int measure_drive_time(void)**: Drives forward in a loop that counts upward from zero. Returns count value when collision sensed.
- **int measure_turn_time(void)**: Turns left in a loop that counts upward from zero. Returns count value when collision sensed.
- **void draw_square(int side_time, int turn_time)**: Waits for all toggle switches to be turned on. Draws a square using the provided times.
- **void delay(void)**: Delays approximately one-half second.

Your main program must measure the drive time, the turn time, and then draw a square. When it has finished the square, it must stop and beep five times using one-half second on, one-half second off. When beep on, all LEDs are also on. When beep off, all LEDs are also off. When it finishes beeping, it enters an infinite loop until reset.

Submissions

- Well-commented VHDL code is expected.
- Project archives are created using Project → Archive Project... in Quartus.
- Archive your Quartus project with the name:

lastname-firstname-system.qar

For example, my file would be named livingston-adam-system.qar

- Upload your Quartus archive by the **due date** to the folder listed on the Canvas Assignment so that Dr. Meier can examine your work on his machine.
- You must demonstrate your single-cycle DE10-Lite computer no later than Friday of Week 15.
- Demonstrations will not be accepted during Final Exam week without prior permission.